

TinyURL — Functional Requirements

1. URL Shortening

Given a long URL, the system must generate a unique, shorter alias (short code).

2. Redirection

Accessing the short URL must redirect the user (HTTP 301/302) to the original long URL.

3. Custom Alias (optional)

Users may supply a custom alias instead of a system-generated one.

4. Link Expiration (optional)

Users can set an expiration date/time after which the short link becomes invalid.

5. Click Analytics

Track number of clicks, timestamp, referrer, device/browser and geolocation per short URL.

6. User Account & Link Management

Authenticated users can view, edit and delete the URLs they created.

7. Duplicate Handling

If the same long URL is submitted again (by the same user), return the existing short URL rather than creating a duplicate.

8. Rate Limiting

Prevent abuse by limiting the number of URLs a single user/IP can create per time window.

TinyURL — Non-Functional Requirements

1. High Availability

The redirection service must be available 99.99% of the time (URL shortening can tolerate slightly lower availability).

2. Low Latency

Redirection must be fast, ideally under 100ms, since it sits on the critical read path.

3. Scalability

System must horizontally scale to support billions of URLs and tens of thousands of requests per second.

4. Durability / Reliability

A created short URL must never be lost; no data loss even under partial system failure.

5. Read-Heavy Optimization

System should be optimized for reads; expected read:write ratio is roughly 100:1.

6. Unpredictability

Short codes should not be sequential or easily guessable to prevent enumeration attacks.

7. Security

Protect against malicious long URLs (phishing/malware), and secure user accounts with proper auth.

8. Consistency

Eventual consistency is acceptable for analytics; strong consistency required for short-code uniqueness.

TinyURL — Back-of-the-Envelope Estimation

Assumptions

- 500M new short URLs created per month
- Read:Write ratio = 100:1 → ~50B redirects/month
- Data retained for 5 years
- Average object size ≈ 500 bytes (URL + metadata)

Write QPS

$500,000,000 / (30 \times 24 \times 3600)$
≈ 193 URLs/sec (average)
Peak (×3) ≈ 580 URLs/sec

Read (Redirect) QPS

$193 \times 100 \approx 19,300$ req/sec (average)
Peak (×3) ≈ 58,000 req/sec

Storage (5 years)

$500\text{M}/\text{mo} \times 60 \text{ months} \times 500\text{B}$
≈ 15B records × 500B
≈ 7.5 TB total

Incoming Bandwidth (writes)

$193 \times 500\text{B} \approx 96.5$ KB/sec

Outgoing Bandwidth (redirects)

$19,300 \times 500\text{B} \approx 9.65$ MB/sec

Cache Memory (80/20 rule)

20% of daily reads are hot
 $19,300 \times 86400 \times 0.2 \times 500\text{B}$
≈ 167 GB → round up to ~200 GB

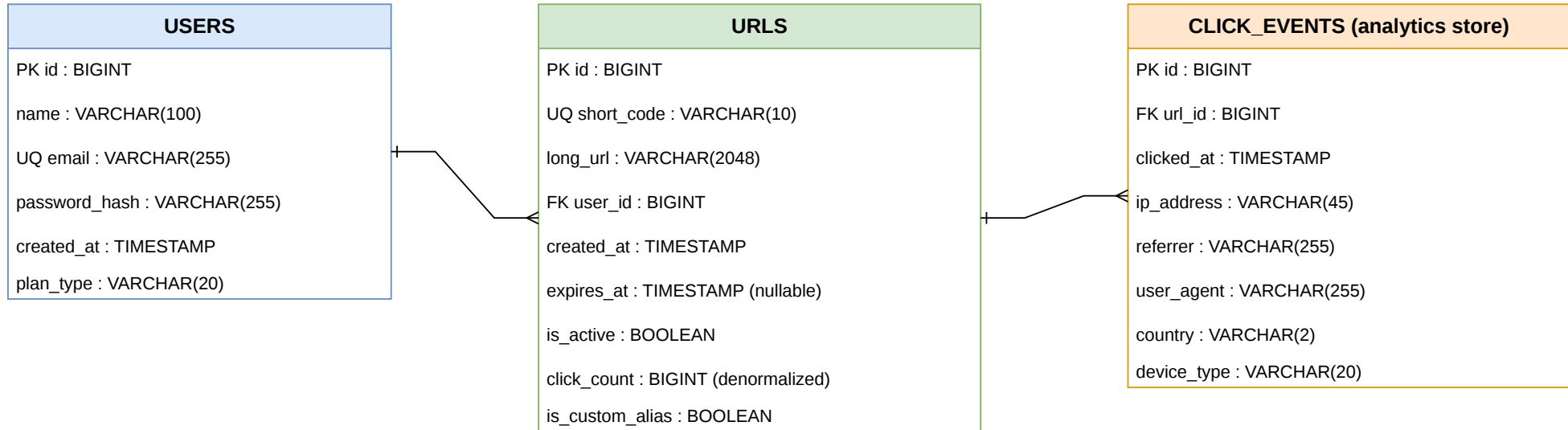
Key-space Sizing

Base62 alphabet [a-zA-Z0-9], 62 symbols
7 chars → $62^7 \approx 3.5$ trillion unique codes
Comfortably covers 15B+ records with headroom

Server Estimate

Assume 1 app server handles ~1,000 req/sec
Peak 58,000 req/sec → ~58 app servers
(plus redundancy across regions/AZs)

TinyURL — Database Design (ER Diagram)



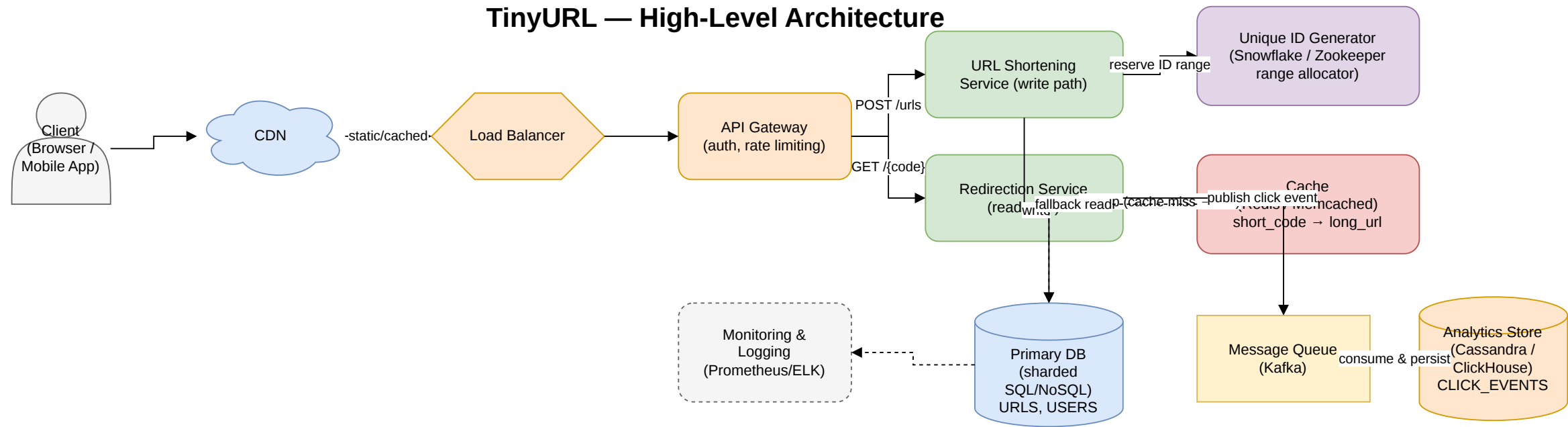
Notes:

- URLS table: relational store (e.g. PostgreSQL/MySQL) sharded by short_code hash for scale.
- CLICK_EVENTS: high write-volume, append-only — better suited to a wide-column / NoSQL store (e.g. Cassandra, DynamoDB) or a Kafka → OLAP pipeline (e.g. ClickHouse).
- click_count on URLS is a denormalized counter updated asynchronously for fast reads.

TinyURL — API Design

POST	<code>/api/v1/urls</code> Create a new short URL. Body: { long_url, custom_alias?, expires_at? } → 201 { short_url, short_code }
GET	<code>/shortCode</code> Redirect to the original long URL. Returns 302 Found with Location header, or 404 if not found/expired.
GET	<code>/api/v1/urls/{shortCode}</code> Fetch metadata for a short URL (long_url, created_at, expires_at, click_count). Requires ownership/auth.
PUT	<code>/api/v1/urls/{shortCode}</code> Update the destination URL or expiration date of an existing short link (owner only).
DELETE	<code>/api/v1/urls/{shortCode}</code> Delete / deactivate a short URL (owner only) → 204 No Content.
GET	<code>/api/v1/urls/{shortCode}/analytics</code> Get click analytics (total clicks, clicks over time, top referrers/countries) for a short URL.
GET	<code>/api/v1/users/{userId}/urls</code> List all short URLs created by a given user, paginated (?page=, ?limit=).
POST	<code>/api/v1/auth/register</code> Register a new user account. Body: { name, email, password }.
POST	<code>/api/v1/auth/login</code> Authenticate a user and return a JWT / session token.
Notes: All endpoints (except redirect & login/register) require Authorization: Bearer . Standard error format: { error_code, message }. Rate-limited via API Gateway (per-user & per-IP).	

TinyURL — High-Level Architecture



Write path: Client → CDN → LB → API Gateway → Shortening Service → ID Generator (unique code) → Primary DB.
Read path: Client → CDN → LB → API Gateway → Redirection Service → Cache (hit) or Primary DB (miss, then populate cache) → 302 redirect.
Click events are published async to a queue and persisted to a separate analytics store to keep the read path fast.